Efficient Route Planning Techniques
Summer Semester 25

3. Exam Task

The following task is part of the final project which has to be submitted until August 20, 2025. To submit your project, create a repository on `https://gitlab.inf.uni-konstanz.de/` and give access to sabine.storandt. Upload your code and a result file (e.g. a PDF or a md) that contains all the requested experimental results, plots and visualizations. DO NOT UPLOAD ROAD NETWORK INSTANCES! Include a README file in your repository which explains how to run your code and how to use any other features. Send an email to sabine.storandt@uni-konstanz.de with the subject 'route planning exam' with which you confirm that your project is submitted and ready to be graded (the email should arrive prior to midnight on August 20).

(1) Application

Choose an application of CH or CCH and use your implementation as basis to make it work. Here are some suggestions, but feel free to do something completely different:

- Make an interactive web application that allows to select the start and end node on a map and then computes the shortest path (via CH) and draws it on the map.
- Engineer the edge weights of the graph to cater for certain user profiles (for example, to produce curvy routes for motorcyclists) and allow for fast updates after a profile choice via CCH.
- Obtain a valid Hub Labeling from the CH and show that it offers even faster query times.
- Compute alternative routes using via-paths over nodes that are settled in the forward and backward run of the CH query.
- Assume some of the nodes are POIs (for example, bars) and find the shortest path from s to t that visits at least one such POI (via CH).
- Allow the user to block some edges and find the shortest path (via CCH) that only traverses unblocked edges.
- ...

Describe your application, the way you realized it, and – if applicable – the respective experimental results.

(2) Grading of the Project

The following criteria will be considered:

- Correctness and soundness of the code. The code should be readable (sensible variable names, comments where necessary) and the running times should be reasonable.
- The requested evaluations should be included in the result file in a comprehensible manner.
- The results should be convincing. For example, the number of shortcuts in the CH and CCH should not be too large (about a factor 2-3 over the number of original edges), the customization times should be small (order of a few seconds), and the (C)CH-Dijkstra should be faster than Dijkstra on the original graph.
- Realization of the application task (creativity, implementation, documentation).